

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CAMPUS CORNÉLIO PROCÓPIO
BACHARELADO EM ENGENHARIA DE SOFTWARE – ES44B

JOSIANE MARIANE BATISTA
MARIA CLARA NASCIMENTO DE JESUS
PAMELA BERTI BRAZ

COMPLEMENTE

CORNÉLIO PROCÓPIO
JUNHO, 2025

Josiane Mariane Batista

Maria Clara Nascimento de Jesus

Pamela Berti Braz

COMPLEMENTE

Projeto elaborado na disciplina de Programação Orientada a Objetos 2 do curso de Engenharia de Software, do Campus Cornélio Procópio da Universidade Tecnológica Federal do Paraná.

Professora: Gisele Alves Santana

CORNÉLIO PROCÓPIO

JUNHO, 2025

RESUMO

O projeto Complemente consiste em uma aplicação desktop desenvolvida em Java com o objetivo de auxiliar os estudantes da UTFPR na gestão das atividades complementares, centralizando o registro e acompanhamento dessas atividades ao longo do curso. A metodologia utilizada baseou-se na modelagem UML, adoção dos padrões de projeto MVC e DAO, uso da IDE NetBeans e banco de dados relacional MySQL. O sistema permite cadastro, consulta e geração de relatórios, organizando as ACs conforme exigências institucionais. Como principal resultado, obteve-se uma aplicação funcional com controle por grupo e geração automática de relatório final. Conclui-se que os objetivos foram atingidos, com possibilidade de melhorias futuras, como migração para ambiente web e personalização por curso.

Palavras-chave: Atividade complementar, Hora complementar, UTFPR, Java.

SUMÁRIO

1	INTRODUÇÃO	1
2	FERRAMENTAS E TECNOLOGIAS	7
3	DESENVOLVIMENTO	9
3.1	Levantamento dos Requisitos	10
3.2	Diagrama de Casos de Uso	12
3.3	Diagrama de Classes	13
3.4	Boas Práticas	14
3.5	Banco de Dados	16
3.6	Telas do sistema	17
3.7	Link do repositório	20
4	CONCLUSÕES	21
	REFERÊNCIAS	23
	APÊNDICES	25

1 INTRODUÇÃO

O projeto Complemente propõe o desenvolvimento de um sistema desktop para o melhor gerenciamento e armazenamento das atividades complementares realizadas pelos estudantes da Universidade Tecnológica Federal do Paraná (UTFPR), campus Cornélio Procopio. Isso se justifica pelo fato de que a maioria dos cursos de graduação ofertados nas universidades brasileiras, com exceção dos cursos tecnológicos, exige o cumprimento de uma carga horária de Atividades Complementares (AC), conforme estabelecido pelo Ministério da Educação (MEC). Essas atividades têm como principal objetivo contribuir para o desenvolvimento social, acadêmico e profissional dos alunos, ampliando sua formação para além dos conteúdos obrigatórios da matriz curricular.

Segundo Bussolotti, Oliveira, Pires e Veiga (2016), as atividades complementares têm como principal finalidade enriquecer os projetos pedagógicos dos cursos de ensino superior, promovendo a aproximação dos estudantes com sua futura profissão, além de fortalecer a relação entre a instituição de ensino e a comunidade. Mais do que simplesmente somar horas à grade curricular, as atividades complementares são consideradas instrumentos essenciais para o desenvolvimento de competências profissionais. Indo de acordo com o proposto pela UTFPR-CT (2018), que frisa o currículo no ensino superior como algo que vai além das atividades convencionais de sala de aula, incluindo iniciativas como programas acadêmicos de formação ampla, projetos de extensão, visitas técnicas, programas de iniciação científica e tecnológica, bem como atividades culturais, políticas e sociais. O objetivo central dessas ações é ampliar a formação sociocultural e profissional dos alunos, preparando-os de forma mais completa para os desafios do mercado de trabalho.

Considerando a necessidade de organizar e validar as ACs ao longo dos períodos letivos, diversas universidades, juntamente com seus estudantes e colaboradores, passaram a adotar ou desenvolver sistemas para o armazenamento, controle e validação dessas atividades. De acordo com Nascimento (2023), a maioria dessas soluções é integrada a ambientes virtuais de aprendizagem ou a portais acadêmicos institucionais, possibilitando ao aluno o registro das atividades realizadas, o envio de documentos comprobatórios e o acompanhamento do status

de validação junto à coordenação de curso. Essa centralização de processos visa garantir maior transparência, agilidade e segurança na gestão das informações acadêmicas.

Entretanto, é importante destacar que muitas Instituições de Ensino Superior (IES) ainda recorrem a métodos manuais ou semiautomatizados para gerenciar as etapas de validação das atividades complementares. Conforme apontado por Nascimento (2023), é comum que essas instituições exijam a entrega física de comprovantes, o preenchimento de formulários manuscritos ou a realização de entregas digitais por e-mail ou em plataformas de ensino a distância. Esses processos geram diversos transtornos para os estudantes, como a perda ou rasura de documentos já entregues, a falta de informações claras sobre as ACs homologadas, a demora na confirmação de recebimento e homologação, além da necessidade de comparecimento presencial na instituição. Há também dificuldade de acesso à lista de atividades aceitas como ACs por cada curso, o que aumenta ainda mais a insegurança dos alunos.

Além disso, a necessidade de auxílio aos estudantes no processo de registro e cumprimento das ACs é reforçada em uma publicação do Jornal Unoeste, feita por Aguiar Filho, Dutra e Costa (2024), que evidencia o quanto os alunos sentem dificuldade para entender, localizar e realizar as atividades exigidas. Como resposta a esse cenário, os autores apresentam o aplicativo “Universitário do Bem”, desenvolvido com o propósito de auxiliar na identificação e execução das atividades complementares, criando um ecossistema universitário de oportunidades voltadas para essa finalidade. Essa solução, voltada à orientação e incentivo à participação dos estudantes em ações extracurriculares, ilustra como o suporte tecnológico pode facilitar esse processo.

Diante dessas limitações dos métodos tradicionais, um dos grandes desafios enfrentados pelas universidades é tornar o processo de gestão das ACs mais simples, acessível e transparente. Incluindo uma maneira em que o acompanhamento seja realizado em tempo real, de forma totalmente digital e com visibilidade da totalidade das atividades já entregues e homologadas, além de proporcionar fácil acesso às regras de atividades complementares aceitas nos cursos.

Quando se trata especificamente da gestão formal e documental das ACs, algumas instituições optam por sistemas mais estruturados e integrados. Exemplo

disso é o Sistema Integrado de Gestão de Atividades Acadêmicas (SIGAA), amplamente utilizado em instituições federais, como citado por Nascimento (2023). Esse sistema oferece aos alunos um ambiente web centralizado, que permite o envio de documentos, o acompanhamento das validações e a consulta de pendências de forma contínua e automatizada, evidenciando um alto nível de integração entre os setores administrativos e acadêmicos. No entanto, nem todas as instituições de ensino superior utilizam plataformas tão consolidadas. Muitas optam pelo desenvolvimento de sistemas próprios, com diferentes níveis de automação e recursos.

Um exemplo é o Instituto Federal do Rio Grande do Sul, campus Porto Alegre, que não dispunha de uma ferramenta específica para gestão das horas complementares (BERNARDONI, 2023). Em função dessa lacuna, Bernardoni (2023) propôs uma plataforma web desenvolvida com tecnologias Java e JavaServer Faces, cujo objetivo é gerir, intermediar e validar os processos de aproveitamento das atividades complementares, integrando alunos, professores responsáveis e coordenadores em um único ambiente digital.

Seguindo essa mesma linha de soluções personalizadas, Toledo e Sirqueira (2021) descrevem o desenvolvimento de um aplicativo móvel na linguagem JavaScript para o Centro Universitário Academia (UniAcademia), cujo foco é oferecer suporte aos estudantes, coordenadores e supervisores no registro, controle, validação e integralização curricular das atividades complementares. Essa aplicação caracteriza-se por facilitar a comunicação entre os diferentes agentes envolvidos, indo além de um sistema de uso exclusivo do aluno.

Ainda no contexto de aplicações móveis, Nascimento (2023) apresenta uma solução desenvolvida em React Native, também em JavaScript, com foco exclusivo no estudante. Aplicativo este que permite não apenas o cadastro e o gerenciamento das ACs, mas também a criação de categorias, geração de relatórios e exportação de documentos para facilitar a validação junto à instituição. Essa abordagem evidencia o crescimento de soluções com foco direto na experiência individual do aluno.

Ampliando o panorama de alternativas, Lima, Correia e Silva (2018) propuseram o desenvolvimento de um software multiplataforma chamado Akademic, que busca oferecer uma gestão completa das atividades complementares. Essa solução abrange o gerenciamento de usuários, cursos e atividades, a validação dos

registros, a criação de eventos sugeridos para os alunos, entre outras funcionalidades. Sendo um sistema projetado para operar em diferentes dispositivos, como desktops, tablets e smartphones, utilizando tecnologias como Firebase, JavaScript e Angular, o que evidencia a tendência de acessibilidade e integração em múltiplos ambientes tecnológicos.

Especificamente na Universidade Tecnológica Federal do Paraná, campus Cornélio Procopio (UTFPR-CP), para os cursos de Engenharia da Computação e Engenharia de Software, o processo atual de gestão das atividades complementares ainda apresenta limitações. Atualmente, a instituição utiliza um canal dentro da plataforma Moodle, no formato de uma disciplina específica para as ACs. Por meio desse canal, os alunos têm acesso às diretrizes, podem realizar o envio de seus relatórios e anexar documentos comprobatórios. No entanto, essa solução funciona apenas como um repositório de submissões, sem oferecer ao estudante a possibilidade de acompanhamento em tempo real sobre seu progresso, carga horária acumulada ou pendências por grupo de atividade. Além disso, muitos alunos sequer têm conhecimento da existência dessa disciplina no Moodle, descobrindo sua obrigatoriedade apenas em fases avançadas do curso, o que prejudica o planejamento acadêmico ao longo dos semestres.

De acordo com a Instrução Normativa Conjunta COENC-CP e COENS-CP 02/2023, de 10 de abril de 2023, os cursos de Engenharia de Computação e Engenharia de Software da UTFPR-CP exigem um mínimo de 60 pontos em atividades complementares para aprovação final. As atividades estão distribuídas em três grupos distintos: Grupo 1, que contempla ações de formação social, humana e cultural; Grupo 2, com atividades de cunho comunitário e de interesse coletivo; e Grupo 3, relacionado à iniciação científica, tecnológica e à formação profissional. As ações válidas incluem desde participação em palestras e cursos, até envolvimento com projetos sociais e atividades de extensão, com a pontuação variando de acordo com a natureza e a relevância de cada participação.

Por consequência das diferentes normas, grupos e subdivisões, em conjunto ao fluxo de etapas de registro, catalogação, verificação, contagem e validação das atividades, a gestão dessas atividades se torna custosa, sendo necessário o uso de ferramentas auxiliares como planilhas eletrônicas ou sistemas computacionais. Um dos principais problemas está na falta de clareza sobre quais atividades são aceitas para a contabilização de pontos e quantas horas cada uma representa, pois não

sabem como procurar essas informações. Além disso, é comum que os estudantes não acompanhem adequadamente o cumprimento das exigências mínimas e máximas de cada grupo de atividades, o que resulta em inconsistências no momento da conferência final.

Outro fator que contribui para essas dificuldades é a descentralização das informações. Cada curso possui autonomia para definir e gerenciar suas regras e procedimentos, sem uma plataforma centralizada que permita aos alunos consultarem de forma rápida e eficiente suas pendências e o status de suas atividades complementares. Sendo que a organização da documentação também representa um desafio recorrente, pois muitos alunos têm dificuldades para armazenar e localizar os certificados correspondentes às atividades mais antigas, já que não há forma de registrar gradualmente a execução destas, além da necessidade de preparar a documentação final na formatação exigida pela instituição. Por isso, é tão relevante uma plataforma que consiga unificar essas funções de gestão para o acadêmico poder validar as atividades necessárias à sua formação.

Buscando atender às necessidades de organização dos alunos e, também, às diretrizes da disciplina de Programação Orientada a Objetos 2, o sistema Complemente foi desenvolvido como uma aplicação desktop utilizando a linguagem Java. Para o armazenamento seguro e estruturado dos dados, utilizou-se o banco de dados relacional MySQL e, para o ambiente de desenvolvimento, foi escolhido o NetBeans. Já no que se refere à arquitetura do sistema, adotou-se o padrão Model-View-Controller (MVC), a estrutura Data Access Object (DAO) e a aplicação do padrão Singleton na conexão com o banco, visando uma melhor organização, reutilização e manutenção do código.

Já a metodologia aplicada ao desenvolvimento do sistema aproximou-se do modelo incremental. O projeto foi dividido em etapas progressivas: iniciou-se com a modelagem UML, seguida da implementação das classes em Java e da criação das tabelas no banco de dados. Em seguida, foram desenvolvidas as interfaces gráficas e feita a integração entre os módulos. A equipe utilizou o Trello para organizar as tarefas com base nos casos de uso, realizou testes pontuais a cada funcionalidade implementada e utilizou esboços em papel como forma de prototipação inicial da interface. Ainda que o plano de execução tenha se mantido estável durante o

processo, foram feitas pequenas adaptações conforme as limitações técnicas surgiram.

Considerando o cenário exposto, tecnologias e metodologias aplicadas na execução, o projeto Complemente tem como objetivo centralizar o registro das atividades complementares dos alunos da UTFPR, campus Cornélio Procopio. Efetivando isso ao longo de todo o período acadêmico do aluno, em uma única aplicação desktop. Sendo que o sistema permitirá o acompanhamento contínuo do progresso em cada grupo de atividades, sinalizando ao usuário quando as cargas mínimas e máximas forem atingidas. Bem como, proporcionará a geração automática de relatório final, no modelo solicitado pela instituição e curso. Visando assim, proporcionar melhor gestão para os alunos, facilidade para os professores responsáveis, maior índice de aprovação na disciplina e menor índice de falha nos relatórios finais.

2 FERRAMENTAS E TECNOLOGIAS

Inicialmente, para projetar a ideia, utilizaram-se as práticas da *Unified Modeling Language* (UML), em português Linguagem de Modelagem Unificada, para a criação de diagramas que caracterizam o projeto e orientam a programação. Com base nos requisitos levantados no início do escopo do projeto, foram elaborados um diagrama de caso de uso, que permite visualizar as funcionalidades que o sistema deve proporcionar ao usuário, e um diagrama de classes, que detalha as classes de objetos envolvidas no sistema, sejam usuários ou artefatos. Ambos os diagramas foram desenvolvidos com a ferramenta Astah, por oferecer suporte à conexão entre os elementos dos diagramas e disponibilizar uma versão gratuita para os usuários da universidade.

Com essa base pronta, deu-se início à programação, utilizando a linguagem Java. Conforme destacado por Gosling e McGilton (1996), no próprio site oficial da linguagem, esta trata-se de uma tecnologia orientada a objetos desde a sua concepção, consolidada no mercado e amplamente utilizada no desenvolvimento de aplicações desktop. Por esse motivo, a linguagem se adequa ao projeto Complemente, que tem como foco uma aplicação desktop, conforme solicitado pela disciplina. Essa escolha permitiu à equipe aplicar conceitos como encapsulamento e polimorfismo de forma prática no desenvolvimento do sistema. Além disso, o Java oferece boa integração com bancos de dados relacionais, por meio da tecnologia *Java Database Connectivity* (JDBC), o que facilitou a implementação das funcionalidades de armazenamento e recuperação de dados necessárias ao projeto.

Como ambiente de desenvolvimento integrado (IDE), foi utilizada a plataforma NetBeans, devido à sua integração nativa com a linguagem Java e, principalmente, por oferecer suporte facilitado à criação de interfaces gráficas por meio de JFrames. Além disso, a IDE disponibiliza recursos visuais que agilizam o processo de desenvolvimento de aplicações desktop, permitindo o posicionamento e a configuração dos componentes gráficos de forma prática e intuitiva. Características estas que foram fundamentais para a equipe, considerando que o projeto Complemente foi idealizado com telas interativas, seguindo o modelo de um sistema desktop. Adicionalmente, o NetBeans oferece ferramentas de depuração,

gerenciamento de projetos e integração com bancos de dados, o que contribui diretamente para a organização e eficiência durante o desenvolvimento.

Paralelamente, para o armazenamento das informações cadastradas no sistema, foi adotado um Sistema Gerenciador de Banco de Dados (SGBD) relacional baseado em SQL, sendo a ferramenta escolhida o MySQL. A decisão se deu tanto por familiaridade da equipe quanto pelas qualidades profissionais da tecnologia, que a tornam adequada para aplicações acadêmicas e corporativas. Entre suas principais características destacam-se a facilidade de uso, a confiabilidade comprovada e a boa integração com aplicações Java por meio do JDBC. Como apontado por Erickson (2024), o MySQL é um dos bancos de dados mais utilizados mundialmente e oferece conectividade estável, bom desempenho e recursos vastos, sendo ideal para as necessidades do sistema proposto. Para o projeto Complemente, ele possibilitou uma estrutura sólida para registrar, consultar e relacionar as atividades complementares dos alunos, garantindo consistência e segurança na manipulação dos dados.

Por fim, para versionamento de código-fonte e controle das versões durante o desenvolvimento do sistema Complemente, a equipe utilizou a plataforma GitHub, por sua ampla adoção no meio acadêmico e profissional, além de suas funcionalidades para armazenamento, colaboração em equipe e rastreamento de alterações. A gestão de tarefas e a comunicação entre os integrantes foram realizadas de forma prática por meio do WhatsApp, para mensagens rápidas, e do Trello, para o acompanhamento visual das atividades, organização de cronogramas e definição de prioridades. Quanto à elaboração da documentação teórica e acadêmica, foram utilizadas as ferramentas Microsoft Word e Google Docs, que ofereceram recursos de edição colaborativa, controle de versões e facilidade no processo de formatação segundo os padrões exigidos pela instituição.

3 DESENVOLVIMENTO

O processo de desenvolvimento teve início com a modelagem do projeto, utilizando a linguagem UML para traçar os objetivos, definir os requisitos funcionais e não funcionais e criar os diagramas que serviram como base para a implementação. Foram elaborados diagramas de caso de uso e de classes, representando graficamente os atores, artefatos, seus atributos e as funcionalidades que os conectam de forma sistemática. Juntamente aos diagramas, foram rascunhados protótipos de baixa fidelidade em papel, com o objetivo de visualizar a estrutura inicial das interfaces gráficas do sistema proposto.

Com os diagramas prontos, iniciou-se a implementação em código, tendo como primeiro passo a transformação dos atores e artefatos em classes Java, já organizadas segundo o padrão de projeto MVC, para estruturar a distribuição dos arquivos. Foram criadas, inicialmente, as classes do pacote Model, representando as entidades do sistema: Aluno, Atividade, Caracteristica, TotalAtividade e Relatorio. Em paralelo, foram definidas e implementadas as tabelas no banco de dados MySQL, correspondentes a essas entidades, com a definição de atributos e relações conforme o modelo lógico diagramado.

Posteriormente, foram iniciadas as interfaces gráficas por meio da criação dos JFrames no pacote View, com foco funcional, contendo botões, rótulos e campos de texto suficientes para testes preliminares, ainda sem preocupação com aspectos visuais ou usabilidade. Na sequência, foram implementadas as classes de controle de acesso aos dados, localizadas no pacote Controller, responsáveis por estabelecer a comunicação entre as interfaces gráficas, o banco de dados e as classes do modelo. Após a integração entre os módulos, as interfaces foram aprimoradas visando melhor usabilidade, e as funcionalidades de geração de relatórios foram desenvolvidas, permitindo consolidar e apresentar as informações registradas pelos usuários de forma organizada.

3.1 Levantamento dos Requisitos

Para transformar uma ideia em um sistema real, é necessário definir claramente o que esse sistema deve realizar e como ele deve se comportar. Dentro dos conceitos de engenharia de software, essa definição ocorre por meio dos requisitos funcionais, que descrevem as funcionalidades específicas que o sistema deve oferecer para atender às necessidades dos usuários e aos objetivos do projeto. Esses requisitos representam ações e comportamentos esperados do sistema, como operações CRUD (*Create, Read, Update e Delete*), funcionalidades administrativas ou cálculos automáticos. São fundamentais para nortear a implementação das funcionalidades de acordo com os objetivos do sistema. Os requisitos funcionais que definem o sistema Complemente encontram-se exemplificados no Quadro 1.

Quadro 1 – Requisitos funcionais

Identificador	Descrição	Prioridade
RF01	O sistema deve permitir o cadastro do estudante com RA, nome, senha, e-mail institucional, nome do curso e código do curso.	Alta
RF02	O sistema só deve permitir que o usuário faça login se utilizado RA e senha válidos.	Alta
RF03	O sistema deve permitir as funcionalidades CRUD para o usuário em seu próprio perfil.	Média
RF04	O sistema deve permitir o cadastro de atividades complementares com tipo, descrição, início, término e duração.	Alta
RF05	O sistema deve permitir as funcionalidades CRUD para atividades complementares.	Alta
RF06	O sistema deve fornecer a lista de tipos de atividade.	Baixa
RF07	O sistema deve identificar o grupo a qual a atividade se refere pelo seu tipo.	Alta
RF08	O sistema deve calcular os pontos a partir dos dados da atividade cadastrada.	Alta
RF09	O sistema deve exibir em formato de dashboard as atividades cadastradas pelo usuário.	Baixa
RF10	O usuário deve poder selecionar quais atividades de cada grupo vai para o relatório.	Média
RF11	O sistema deve indicar a quantidade de horas que faltam para atingir o mínimo de cada grupo.	Baixa
RF12	O sistema deve indicar a quantidade de horas que ultrapassaram o máximo de cada grupo.	Baixa
RF13	O sistema deve gerar relatórios em formato PDF quando solicitado pelo estudante.	Média

Fonte: Autoria Própria (2025)

Complementarmente, têm-se os requisitos não funcionais, que indicam como o sistema deve operar. Eles abrangem aspectos relacionados à qualidade, desempenho, segurança, usabilidade e confiabilidade, entre outros, e descrevem as restrições e características técnicas que o sistema deve cumprir, como definições de acessibilidade, formato de dados ou linguagem adotadas. Esses requisitos são

igualmente importantes, pois garantem que o sistema atenda a padrões mínimos de eficiência e estabilidade. No sistema Complemente, foram adotados requisitos dos tipos usabilidade, acessibilidade, implementação, segurança, legislativo, portabilidade e armazenamento, conforme demonstrado no Quadro 2.

Quadro 2 – Requisitos não funcionais

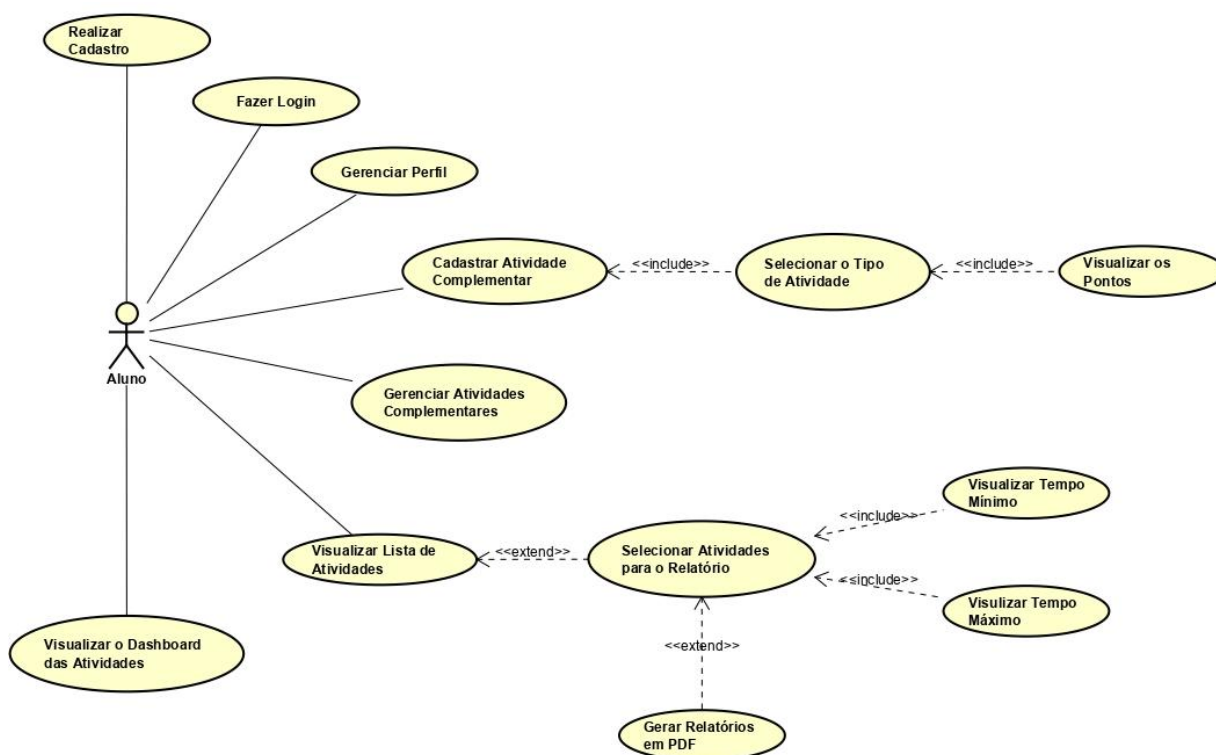
Identificador	Descrição	Tipo
RNF01	O sistema deve gerar o relatório em apenas um clique.	Usabilidade
RNF02	O sistema deve possuir interface acessível.	Acessibilidade
RNF03	O sistema deve ser implementado utilizando a linguagem Java.	Implementação
RNF04	O sistema deve ser gerenciado a partir da linguagem SQL.	Implementação
RNF05	O sistema deve validar e-mails no formato padrão: usuario@alunos.utfpr.edu.br.	Segurança
RNF06	O sistema deve validar RAs no formato numérico de 7 dígitos: 1234567.	Segurança
RNF07	O sistema deve criptografar todos os dados conforme a Lei Nº 13.709/2018 (LGPD).	Legislativo
RNF08	O sistema deve funcionar corretamente nos sistemas operacionais Windows e Linux.	Portabilidade
RNF09	O sistema deve garantir que os dados de cada usuário permaneçam privados e acessíveis apenas com autenticação válida.	Segurança
RNF10	O sistema deve armazenar os dados e atividades do usuário em um arquivo local na máquina.	Armazenamento
RNF11	O sistema deve garantir que os dados de um usuário não sejam acessíveis por outro usuário.	Segurança

Fonte: Autoria Própria (2025)

3.2 Diagrama de Casos de Uso

O diagrama de caso de uso apresenta uma visão do comportamento do sistema, evidenciando como os atores interagem com as funcionalidades definidas pelos requisitos funcionais. Esse tipo de diagrama utiliza conexões como *include* e *extend* para representar ações opcionais ou dependentes, além de ligações diretas para as ações que o usuário deve ser capaz de realizar. O diagrama de caso de uso do sistema Complemente, ilustrado na Figura 1, demonstra os requisitos já estruturados como casos de uso e sua relação com o ator principal, que neste contexto é o Aluno.

Figura 1 – Diagrama de casos de uso

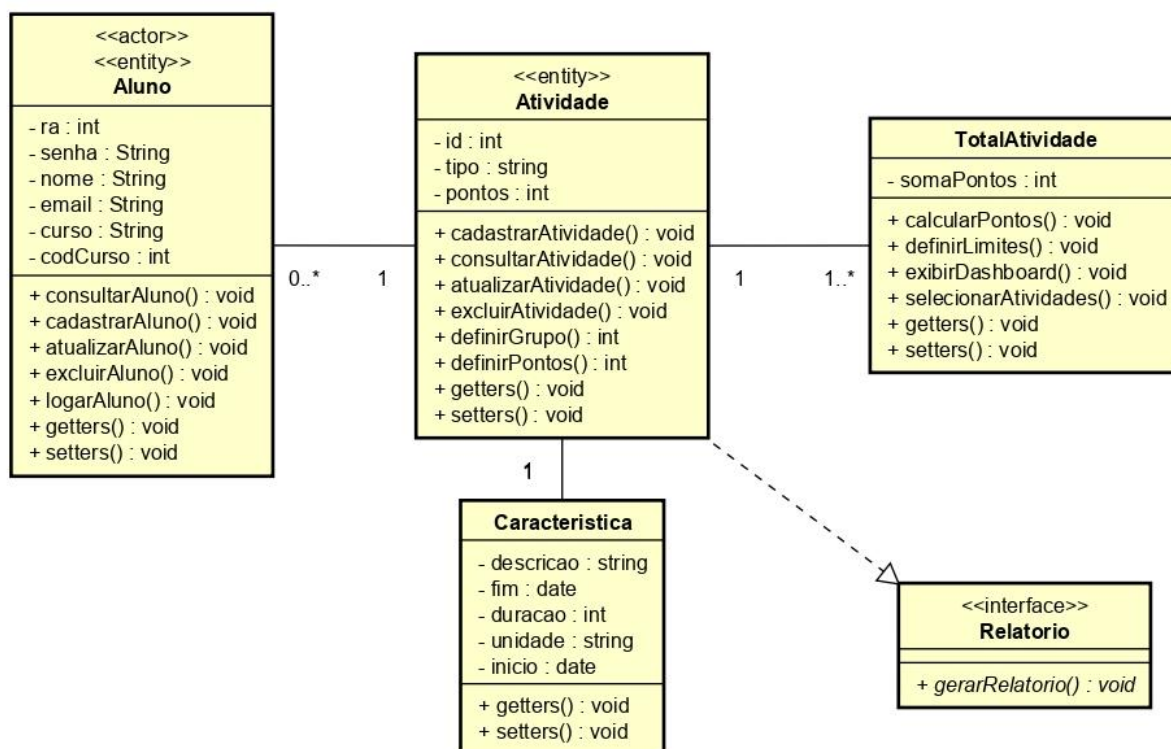


Fonte: Autoria Própria (2025)

3.3 Diagrama de Classes

O diagrama de classes está diretamente ligado à modelagem orientada a objetos, sendo, nesse contexto, uma das representações mais relevantes para a estruturação do projeto. Apesar disso, é considerada uma boa prática iniciar a modelagem pelo diagrama de casos de uso, pois, a partir dele, torna-se possível compreender as interações entre as entidades e, com base nos requisitos, identificar os atores e artefatos que darão origem às classes, com seus respectivos atributos e métodos (operações), os quais estão diretamente relacionados aos casos de uso definidos. Esse diagrama funciona como o principal guia para a criação das classes na linguagem Java, bem como para o mapeamento das tabelas do banco de dados relacional. No projeto Complemente, foram modeladas cinco classes principais, com seus atributos e métodos correspondentes, sendo uma delas a interface Relatorio, que não possui atributos e apenas um método, conforme ilustrado na Figura 2.

Figura 2 – Diagrama de classes

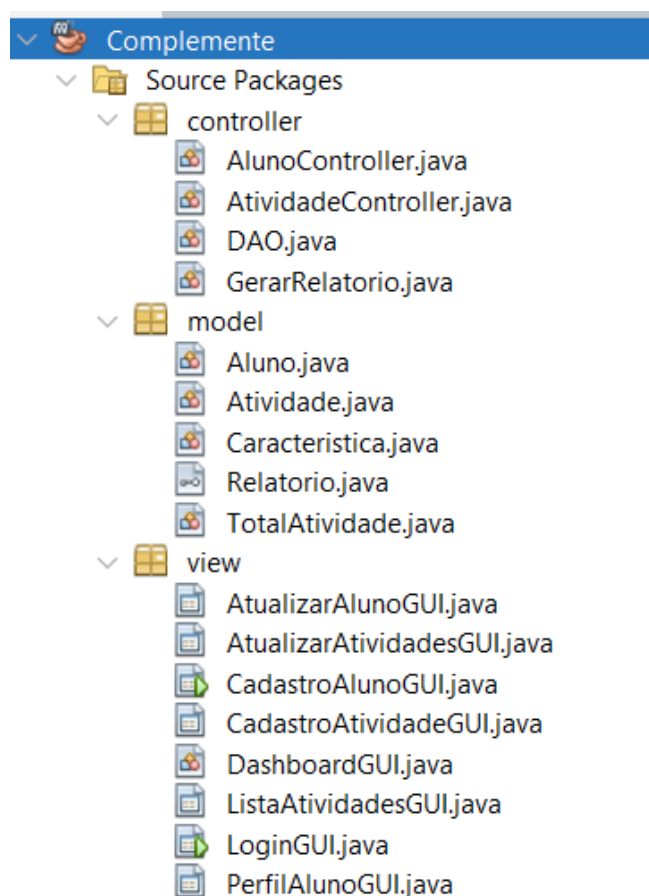


Fonte: Autoria Própria (2025)

3.4 Boas Práticas

Durante o desenvolvimento do sistema Complemente, foram aplicadas boas práticas de organização de código e estrutura de projeto, destacando-se o uso dos padrões de projeto MVC (*Model-View-Controller*) e DAO (*Data Access Object*). O padrão MVC foi adotado com o objetivo de separar a aplicação em três camadas principais, o que facilita tanto o desenvolvimento quanto a manutenção do código. A camada Model representa a lógica de negócio e os dados principais do sistema, como as entidades Aluno, Atividade e Relatório. A camada View é responsável apenas pela exibição dos dados ao usuário, sendo composta pelas interfaces gráficas desenvolvidas com JFrames. Já a camada Controller atua como intermediária entre o Model e a View, manipulando os dados e gerenciando a conexão com o banco de dados. Essa estruturação é ilustrada na Figura 3, que apresenta o padrão aplicado ao projeto.

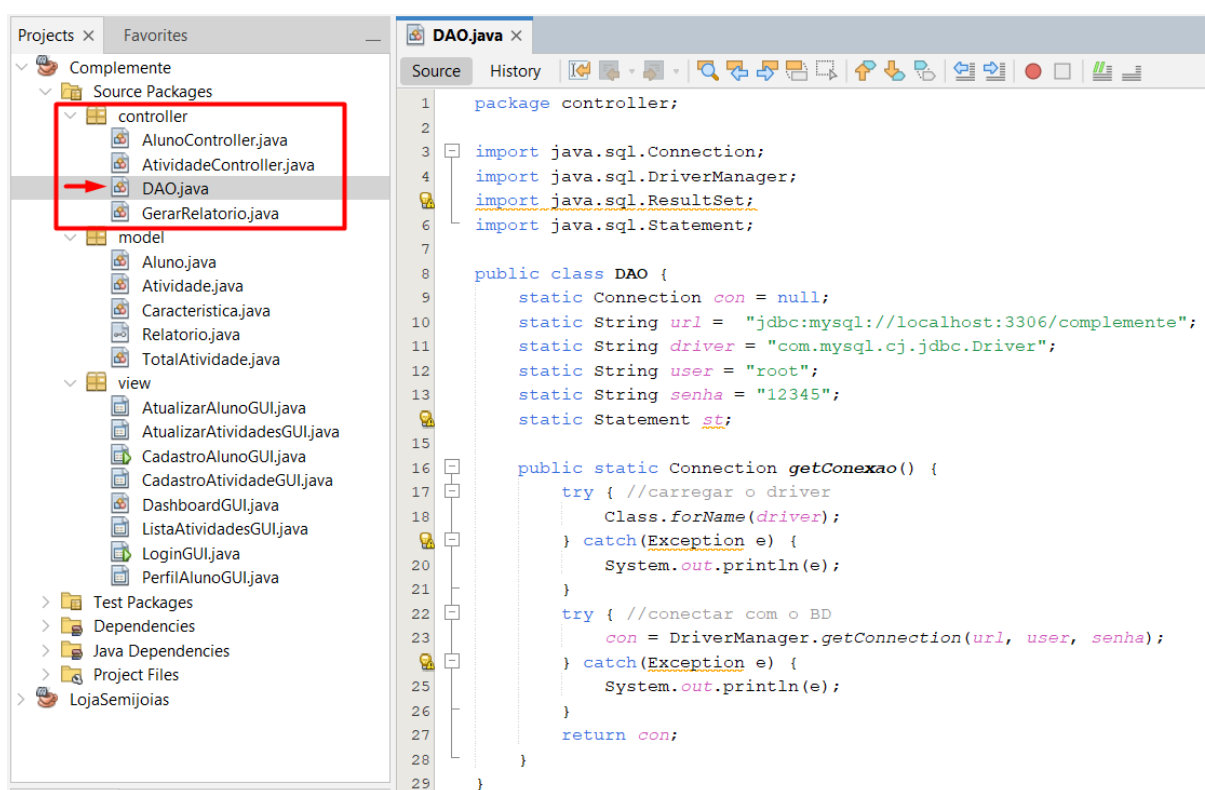
Figura 3 – Pacotes em padrão MVC



Fonte: Autoria Própria (2025)

Além disso, foi utilizado o padrão DAO (*Data Access Object*), que tem como objetivo isolar a lógica de acesso a dados da lógica de negócio. Dessa forma, as entidades Aluno e Atividade possuem classes Controller correspondentes, responsáveis por realizar as operações de acesso ao banco de dados, como inserção, consulta, atualização e remoção de registros (CRUD). Essas operações são realizadas por meio de uma conexão com a classe DAO, dentro do pacote Controller. A aplicação deste padrão no sistema Complemente centraliza a responsabilidade pelas conexões com o banco de dados em uma única classe, que está representada na Figura 4, e também no Apêndice I, que contém o código.

Figura 4 – Classes do padrão DAO



Fonte: Autoria Própria (2025)

3.5 Banco de Dados

O banco de dados escolhido para o sistema Complemente foi do tipo relacional SQL, por permitir a definição clara de entidades, atributos e suas relações, integrando-se bem à modelagem orientada a objetos adotada no projeto. Como plataforma de desenvolvido, utilizou-se o MySQL, selecionada principalmente pela familiaridade da equipe com a ferramenta, por sua interface intuitiva e por oferecer bom desempenho em aplicações de pequeno a médio porte, como é o caso deste sistema. Para o projeto, foram criadas tabelas que armazenam os dados de alunos, atividades, pontuações e demais atributos relevantes ao funcionamento do sistema, como pode ser observado pelo código disponível no Apêndice II. Na tabela de alunos, são registrados os dados de todos os usuários cadastrados naquela instância local do sistema, conforme ilustrado na Figura 5.

Figura 5 – Tabela de dados de alunos

ra	nome	email	senha	codCurso	curso
1234561	Livia Martins	l@alunos.utfr.edu.br	123	62	Engenharia Eletrônica
1234562	Alan Ferreira	alan@alunos.utfr.edu.br	123	43	Engenharia Mecânica
1234563	Beatriz Moura	bia@alunos.utfr.edu.br	123	41	Engenharia de Computação
1234564	Paula Matos	paula@alunos.utfr.edu.br	123	42	Engenharia Elétrica
1234567	João Silva	joao@alunos.utfr.edu.br	12345	46	Licenciatura em Matemática
1234568	Maria Souza	m@alunos.utfr.edu.br	12345	49	Engenharia de Controle e Automação
1234569	Gabriel Almeida	gab@alunos.utfr.edu.br	123	29	Tecnologia em Análise e Desenvolvimento de Sis...
2369885	Josiane Mariane Batista	joasianebatista@alunos.utfr.edu.br	12345	65	Engenharia de Software
2564254	Pamela Berti Braz	pbraz@alunos.utfr.edu.br	1234	65	Engenharia de Software
2601435	Maria Clara Nascimento de Jesus	mariajesus@alunos.utfr.edu.br	12345	65	Engenharia de Software
7654321	Luis Gustavo Moretto	lumoretto@alunos.utfr.edu.br	123	41	Engenharia de Computação
7654322	Leon Costa	leon@alunos.utfr.edu.br	123	49	Engenharia de Controle e Automação

Fonte: Autoria Própria (2025)

Na tabela de atividades, são armazenadas as informações relacionadas às tarefas realizadas pelos alunos. Essa estrutura utiliza chaves estrangeiras (RAs dos alunos) para associar corretamente as atividades às suas respectivas entidades, como demonstrado na Figura 6.

Figura 6 – Tabela de dados de atividades

	id	ra	tipo	descricao	inicio	fim	duracao	unidade	pontos	grupo
▶	2	2601435	Diretório, Centro Acadêmico, Entidade de Class...	Seminários	2025-01-25	2025-07-25	6	meses	10	2
	3	2601435	Estágio não obrigatório	QA	2024-07-22	2025-01-22	20	horas	10	3
	4	2601435	Ouvinte em palestras, congressos e seminários ...	Seminários IA	2024-10-18	2025-10-18	5	horas	5	3
	5	2369885	Curso de língua estrangeira	Curso de Inglês	2024-03-09	2024-09-09	6	meses	10	1
	6	2369885	Trabalho voluntário, atividades comunitárias em...	Atividade comunitária	2024-10-12	2025-04-12	6	meses	10	2
	7	2369885	Estágio não obrigatório	QA	2024-07-22	2025-01-22	20	horas	10	3
	8	2369885	Aprovação em disciplina de enriquecimento curri...	Aulas de Cálculo	2022-03-15	2022-09-15	1	semestres	10	3
	9	2564254	Atividade esportiva	Academia Bem-Estar	2024-03-15	2024-09-15	6	meses	10	1
	10	2564254	Projetos de extensão e de interesse social	Projeto de Extensão IntelGente	2024-03-15	2024-09-15	1	semestres	30	2
	11	2564254	Projetos multidisciplinares ou interdisciplinares	Projetos com mecânica	2024-03-15	2024-09-15	1	semestres	30	3
	12	2564254	Aprovação em disciplina de enriquecimento curri...	Matéria do curso de matemática	2024-03-15	2024-09-15	1	semestres	10	3
	13	2601435	Curso de língua estrangeira	aula de inglês	2025-01-25	2025-07-25	1	semestres	10	1
	14	2601435	Atividades beneficentes	cesta básica	2025-01-25	2025-07-25	6	meses	10	2
	15	2601435	Estágio não obrigatório	Programação	2025-01-25	2025-07-25	20	horas	10	3
	16	2601435	Atividade esportiva	academia	2026-09-20	2027-03-20	6	meses	10	1
	17	2369885	Atividade esportiva	academia	2025-01-12	2025-07-12	1	semestres	10	1

Fonte: Autoria Própria (2025)

3.6 Telas do sistema

Com o código estruturado segundo os padrões de projeto MVC e DAO, aliado à criação do banco de dados e à base sólida de modelagem por meio dos diagramas, iniciou-se a etapa de desenvolvimento das interfaces gráficas. Ao todo, foram construídas nove telas por meio de JFrames, viabilizando a utilização plena do sistema conforme seus objetivos. Algumas das principais interfaces do sistema serão apresentadas nesta seção.

A primeira delas, ilustrada na Figura 7, é a tela de perfil do aluno. Nela, o usuário pode visualizar seus dados pessoais, previamente cadastrados no sistema, além de acompanhar o resumo das pontuações obtidas em cada grupo de atividades complementares. Essa tela também sinaliza se o aluno está apto a gerar o relatório final e, assim, concluir a disciplina. Outras funcionalidades podem ser observadas na imagem, como as disponíveis nos botões visíveis na interface.

Figura 7 – Tela de perfil do aluno

A interface 'PERFIL DO ALUNO' apresenta os seguintes elementos:

- Botão 'Sair da conta' no canto superior esquerdo.
- Título 'PERFIL DO ALUNO' no topo central.
- Dados pessoais do aluno:
 - Nome: Pamela Berti Braz
 - RA: 2564254
 - E-mail: pbraz@alunos.utfpr.edu.br
 - Curso: Engenharia de Software - 65
- Botão 'Atualizar dados' ao lado do curso.
- Tabela 'Resumo de Atividades':

	Pontos mínimos	Pontos cumpridos	Aptidão
Grupo 1	10	10	OK
Grupo 2	10	45	OK
Grupo 3	10	30	OK
Total	60	85	OK
- Botões de ação: 'Cadastrar Atividade', 'Visualizar Lista de Atividades' e 'Editar Atividade'.
- Botão 'Excluir Perfil' no canto inferior direito.

Fonte: Autoria Própria (2025)

Outra tela de destaque, essencial para o uso do sistema, é a que apresenta a lista de atividades cadastradas, segmentadas por grupo. Todos os atributos relevantes são exibidos, bem como uma caixa de seleção que permite ao aluno indicar se determinada atividade deve ou não ser incluída no relatório final, conforme demonstrado pela Figura 8. Ao selecionar as atividades e solicitar a geração do relatório, o sistema realiza a verificação dos critérios mínimos e máximos, caso algum deles não seja atendido, uma notificação em tela é exibida. Essa interface também contém um botão para adicionar novas atividades, redirecionando o usuário para a tela de cadastro correspondente.

Figura 8 – Tela de lista de atividades

Lista de Atividades

Grupo 1

Tipo: Atividade esportiva Pontos: 10
Descrição: Academia
Período: 2024-03-12 a 2025-03-12 Qtd. Pontos: 10 ☐ Selecionar
Duração: 2 semestres

Grupo 2

Tipo: Projetos de extensão e de interesse social Pontos: 30
Descrição: Overload
Período: 2023-09-10 a 2024-10-10 Qtd. Pontos: 30 ☐ Selecionar
Duração: 2 semestres

Grupo 3

Tipo: Empresa Júnior, Hotel Tecnológico ou Incubadora Tecnológica Pontos: 30
Descrição: Empresa X
Período: 2023-09-10 a 2024-10-10 Qtd. Pontos: 30 ☐ Selecionar
Duração: 2 semestres

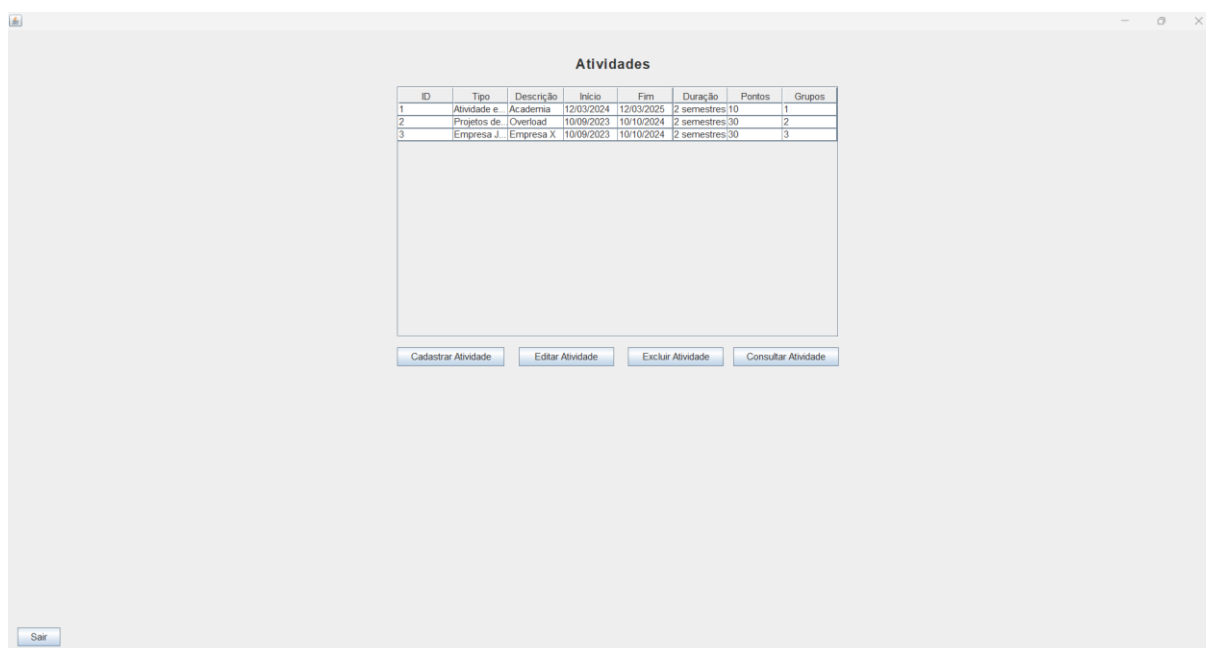
Adicionar Atividade

Voltar Gerar Relatório

Fonte: Autoria Própria (2025)

Além da lista de atividades, o sistema oferece uma tela destinada à consulta objetiva das atividades registradas, sem vínculo direto com a geração de relatórios. Essa tela, apresentada na Figura 9, foca em oferecer ao usuário as operações CRUD de forma mais acessível e direta. Por meio dela, é possível visualizar todas as atividades vinculadas ao perfil do aluno, bem como cadastrar novas, editar, excluir ou consultar detalhes específicos de cada uma.

Figura 9 – Tela de consulta de atividades cadastradas



Fonte: Autoria Própria (2025)

3.7 Link do repositório

Para o versionamento do código, compartilhamento entre os membros da equipe e organização profissional do projeto, foi utilizado a plataforma GitHub. Além disso, o repositório serve como forma de disponibilizar publicamente a solução desenvolvida, permitindo que terceiros acessem, conheçam e avaliem a proposta implementada.

O repositório do projeto Complemente encontra-se disponível pelo link:
<https://github.com/pamms2/Complemente>.

4 CONCLUSÕES

Com base no exposto, pode-se afirmar que os objetivos propostos foram alcançados em sua maior parte. O projeto Complemente centraliza os registros das atividades complementares dos alunos, permitindo que estes gerenciem suas atividades por meio de uma aplicação desktop, conforme foi objetivado pela equipe e solicitado pela disciplina. Além disso, em questões mais específicas, o sistema fornece acompanhamento contínuo do progresso em cada grupo de atividades, por meio das telas `PerfilAlunoGUI.java` e `ListaAtividadesGUI.java`. Nesta última, o objetivo de sinalização de cumprimento dos limites mínimo e máximo de horas é atendido ao tentar gerar o relatório, quando o sistema impede a ação caso o mínimo não tenha sido alcançado ou o máximo tenha sido ultrapassado, conforme configurado, apresentando mensagem na tela.

Outro marco importante do sistema é a geração do relatório final de atividades complementares. Essa funcionalidade foi implementada com sucesso, gerando um documento no formato `.docx`. No entanto, o requisito de exportação em PDF ainda não foi atendido, representando uma necessária melhoria futura. Além disso, atualmente, o sistema exige a presença de um modelo prévio em `.docx` para conseguir gerar o relatório final, o que configura uma limitação, para mitigar esse ponto, será necessária uma investigação por parte da equipe quanto à viabilidade de eliminar essa dependência. Quanto ao formato PDF, a dificuldade encontrada decorreu de conflitos entre bibliotecas Java, com relação ao arquivo `pom.xml`, sendo a sua resolução considerada possível e aparenta média complexidade.

Em relação às interfaces gráficas, o NetBeans oferece praticidade na criação via JFrames, mas impõe limitações estéticas. Por consequência disso, o requisito de baixa prioridade relacionado à exibição de dados em dashboard, foi negligenciado propositalmente, por enquanto. Outra influência desse fator é que algumas telas ainda apresentam desajustes visuais, mas sem prejuízo às funcionalidades, que seguem atendendo ao necessário para testes. Essas correções estéticas representam a última etapa do cronograma do projeto, então é natural que ainda estejam em processo. Outros aprimoramentos incluem a vinculação futura da classe `TotalAtividade` ao Aluno no banco de dados, que realiza a soma dos pontos por grupo, informação relevante a ser exibida no perfil do aluno. Optou-se, por ora, por

manter esses dados apenas no código-fonte em Java, mas sua persistência no banco de dados será considerada em futuras versões.

Em relação à adequação funcional, o sistema atualmente está configurado com base nas diretrizes dos cursos de Engenharia da Computação e Engenharia de Software da UTFPR – campus Cornélio Procópio. Contudo, ele não contempla as particularidades dos demais cursos. Para o futuro, espera-se apoio da universidade com o fornecimento das diretrizes específicas, permitindo personalizar o sistema para cada curso do campus e, posteriormente, até mesmo expandi-lo para os demais campi. Ademais, uma melhoria adicional planejada é a criação de uma aba para anexar certificados que comprovem as atividades cadastradas, o que agregaria praticidade e organização para os usuários.

Já com relação aos aspectos não funcionais, atingiu-se o proposto para com os formatos de dados e linguagens requisitadas, entre outros. Porém, não se concretizou as requisições de criptografia de dados como prevê a Lei Nº 13.709/2018 (LGPD) e de armazenamento em arquivo local da máquina. Configurando esses pontos como melhorias em futuras versões também. Por fim, a evolução mais significativa proposta pela equipe é a migração do sistema para uma aplicação web. Essa transformação tornaria o sistema mais acessível e escalável, além de trazer vantagens como acesso multiplataforma, integração com sistemas institucionais, facilidade de manutenção, atualizações centralizadas, melhor experiência do usuário entre outros. Inclusive, essa nova versão poderia ser integrada ao site oficial da universidade, ampliando seu alcance e utilidade.

REFERÊNCIAS

AGUIAR FILHO, Armando Sérgio de; DUTRA, Frederico Giffoni de Carvalho; COSTA, José Maurício. **Aplicativo Universitário do Bem: desenvolvimento de tecnologia para gestão de atividades complementares em instituições de ensino superior**. Colloquium Socialis, Presidente Prudente, v. 8, p. 1–13, jan./dez. 2024. DOI: 10.5747/cs.2024.v8.s178. Disponível em: <http://journal.unoeste.br/index.php/cs/article/view/4965>. Acesso em: 22 jun. 2025.

BERNARDONI, Ailton da Rocha. **Plataforma web para o gerenciamento de atividades acadêmicas complementares de cursos de graduação**. 2022. Trabalho de Conclusão de Curso (Tecnologia em Sistemas para Internet) – Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul, Campus Porto Alegre, Porto Alegre, 2022. Orientador: Fabio Yoshimitsu Okuyama. Disponível em: <https://repositorio.ifrs.edu.br/handle/123456789/980>. Acesso em: 22 jun. 2025.

BUSSOLOTI, Juliana Marcondes; OLIVEIRA, Márcia Regina de; PIRES, Rosana Giovanni; VEIGA, Susana Aparecida da. **A IMPORTÂNCIA DAS ATIVIDADES COMPLEMENTARES NO PROCESSO DE APRENDIZADO: PERCEPÇÃO DOS ALUNOS DE CURSOS DE EDUCAÇÃO A DISTÂNCIA DA UNIVERSIDADE DE TAUBATÉ**. Taubaté/SP, 2016. 8 p. Disponível em: <https://www.abed.org.br/congresso2016/trabalhos/137.pdf>. Acesso em: 21 jun. 2025.

DEVMEDIA. **Aprendendo Java com JDBC**. Disponível em: <https://www.devmedia.com.br/aprendendo-java-com-jdbc/29116>. Acesso em: 22 jun. 2025.

GOSLING, James; McGILTON, Henry. **The Java Language Environment: Contents. White Paper**. Oracle, maio 1996. Disponível em: <https://www.oracle.com/java/technologies/language-environment.html>. Acesso em: 22 jun. 2025.

JDEV TREINAMENTO. **Linguagem Java e suas características**. Disponível em: <https://www.jdevtreinamento.com.br/linguagem-java-e-suas-caracteristicas/>. Acesso em: 22 jun. 2025.

LIMA, Patrick Gonçalves de; CORREIA, Luiz Henrique; SILVA, Carlos Alexandre Gouvea da. **Implementação de um sistema multi-plataforma para gerenciamento de atividades complementares em cursos superiores utilizando banco de dados noSQL**. Revista de Informática Aplicada, v. 14, n. 2, p. 78–87, 2018. Disponível em: https://seer.uscs.edu.br/index.php/revista_informatica_aplicada. Acesso em: 22 jun. 2025.

NASCIMENTO, Isaac James Mangueira do. **Desenvolvimento de uma aplicação móvel para gerenciamento de atividades complementares em instituições de ensino superior**. 2023. 64 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Software) – Universidade Federal do Ceará, Campus Quixadá, Quixadá, 2023. Orientadora: Antônia Diana Braga Nogueira. Disponível em: <https://repositorio.ufc.br/handle/riufc/73945>. Acesso em: 22 jun. 2025.

ORACLE. **NetBeans IDE – The Smarter and Faster Way to Code**. Oracle. Disponível em: <https://www.oracle.com/ee/tools/technologies/netbeans-ide.html>. Acesso em: 22 jun. 2025.

TOLEDO, Paulo Donizete Alves de; SIRQUEIRA, Tassio Ferenzini Martins. **HORAS COMPLEMENTARES: UM APLICATIVO DE GESTÃO DE ATIVIDADESACADÊMICAS**. 2021. 18 f. TCC (Graduação) - Curso de Engenharia de Software, Centro Universitário Academia, Juiz de Fora, 2021. Disponível em: https://www.researchgate.net/publication/353246767_HORAS_COMPLEMENTARES_UM_APLICATIVO_DE_GESTAO_DE_ATIVIDADES_ACADEMICAS. Acesso em: 22 jun. 2025.

UTFPR-CT. **Atividades Complementares**. Disponível em: <https://www.utfpr.edu.br/cursos/coordenacoes/graduacao/curitiba/ct-sistemas-de-informacao/area-academica/atividades-complementares>. Acesso em: 21 jun. 2025.

APÊNDICES

I – Código da classe DAO

```
package controller;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class DAO {
    static Connection con = null;
    static String url = "jdbc:mysql://localhost:3306/complemente";
    static String driver = "com.mysql.cj.jdbc.Driver";
    static String user = "root";
    static String senha = "12345";
    static Statement st;

    public static Connection getConexao() {
        try { //carregar o driver
            Class.forName(driver);
        } catch (Exception e) {
            System.out.println(e);
        }
        try { //conectar com o BD
            con = DriverManager.getConnection(url, user, senha);
        } catch (Exception e) {
            System.out.println(e);
        }
        return con;
    }
}
```

II – Código que gerou o Banco de Dados

```
create database complemento;  
use complemento;
```

```
CREATE TABLE aluno (  
    ra INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    senha VARCHAR(255) NOT NULL,  
    codCurso INT NOT NULL,  
    curso VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE atividade (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    ra INT NOT NULL,  
    tipo VARCHAR(500) NOT NULL,  
    descricao TEXT,  
    inicio DATE NOT NULL,  
    fim DATE NOT NULL,  
    duracao INT NOT NULL,  
    unidade VARCHAR(20),  
    pontos INT DEFAULT 0,  
    grupo INT,  
    FOREIGN KEY (ra) REFERENCES aluno(ra)  
);
```

```
CREATE TABLE totalAtividade (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    ra INT NOT NULL,  
    grupo INT NOT NULL,  
    somaPontos INT NOT NULL,  
    FOREIGN KEY (ra) REFERENCES aluno(ra)  
);
```

```
CREATE TABLE relatorio (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    ra INT NOT NULL,  
    caminhoArquivo VARCHAR(255),  
    FOREIGN KEY (ra) REFERENCES aluno(ra)  
);
```

```
CREATE TABLE relatorioAtividade (  
    idRelatorio INT,  
    idAtividade INT,  
    PRIMARY KEY (idRelatorio, idAtividade),  
    FOREIGN KEY (idRelatorio) REFERENCES relatorio(id),  
    FOREIGN KEY (idAtividade) REFERENCES atividade(id)  
);
```